Code Artifacts — Vehicle Washing Management System

Cover Page

Table 1

Document identification data

Field	Value
Project	Vehicle Washing Management System (Vehicle Washing / LavaRápido)
Deliverable	Code artifacts (section 2.5.4)
Program	Software Analysis and Development Technologist (ADSO)
Group (ficha)	3145556
Apprentices	Brayan Estiven Patiño Cabrera, Juan Camilo Guilombo Osorio, Merari Urbano
Instructor	Karol Daniela Correa
Training center	Industria
Date	October 16, 2026

The system's code lives in 10 GitHub repositories: the web application, the mobile application, and 8 backend repositories. This document inventories the two clients and explains how to build and run them; the backend is described only as far as the clients need it. Source code is not reproduced.

1. Repository Inventory

Table 2

System repositories (`develop` branch, as of October 15, 2026)

Artifact	Repository	Technology	Commits	Last commit
Web application	Front-end-proyecto-web	Angular 21.2.25, TypeScript 5.9	395	2026-10-16
Mobile application	LavadoVehicular-Movil	React Native 0.86.3, Expo ~57, TypeScript 6.0	173	2026-10-16
Infrastructure	lavarapido-infra	Docker Compose	57	2026-10-16
API Gateway	lavarapido-api-gateway	Spring Boot 4.0.8, Java 21	10	2026-10-14
security-service	lavarapido-security-service	Spring Boot 4.0.8, Java 21	48	2026-10-16
customer-service	lavarapido-customer-service	Spring Boot 4.0.8, Java 21	16	2026-10-16

Artifact	Repository	Technology	Commits	Last commit
booking-service	lavarapido-booking-service	Spring Boot 4.0.8, Java 21	44	2026-10-16
operations-service	lavarapido-operation-service	Spring Boot 4.0.8, Java 21	34	2026-10-16
payment-service	lavarapido-payment-service	ASP.NET Core 8	38	2026-10-16
notification-service	lavarapido-notification-service	Spring Boot 4.0.8, Java 21	33	2026-10-16

Table 3

Client size

Artifact	Source files (`.ts`, `.tsx`, `.html`, `.scss`)	Lines	Screens
Web (`Web/src`)	410	45,220	34 routes
Mobile (`LavadoVehicular-Movil/src`)	260	31,612	30 screens

> *Note.* Counted with `find` and `wc -l`; includes tests and comments.

2. Web Application

The web client is an Angular 21 SPA (Angular, n.d.) with standalone components, one lazy-loaded module per role, and translations into four languages (es, en, fr, pt). Every call goes to `/api/v1`, which the development server proxies to the gateway at `localhost:8080`.

2.1 Structure

Table 4

Folders in `Web/src/app`

Folder	Contents
`core/constants`	`API_BASE_URL = '/api/v1'` (`core/constants/api.ts:4`)
`core/guards`	`authGuard` (session and role) and `guestGuard` (hides login from users who already have a session)
`core/interceptors`	`authInterceptor`: adds the JWT and ends the session on a 401
`core/services`	16 HTTP and state services: `auth`, `booking-api`, `vehicles`, `operations-api`, `payments-api`, `promotions-api`, `loyalty-api`, `notifications`, `preferences`, `role-permissions-api`, `user-admin`, `user-session`, `inspection-api`, `inspection-photos`, `local-storage`, `api`
`core/models`, `core/utils`	DTO interfaces and utilities (for example, API error translation)

Folder	Contents
`features/landing`	Public page: navbar, hero, services, features, contact, footer
`features/auth`	Login, sign-up, and three-step password recovery
`features/client`	8 client pages + `viewmodels/client.viewmodel.ts` (layout)
`features/operator`	8 operator pages
`features/admin`	13 administrator pages + 5 stores (`reservations`, `payments`, `operators`, `schedule`, `business`)
`features/public-report`	Read-only inspection report (`/report/:token`)
`shared/components`, `shared/dialogs`, `shared/pipes`	12 reusable components and 20 dialogs (confirm, assign operator, rate, review payment, export…)
`assets/i18n`	`es.json`, `en.json`, `fr.json`, `pt.json`

2.2 Main Dependencies

Table 5

Web libraries (`Web/package.json`)

Library	Version	Purpose
`@angular/*` (core, router, forms, material, cdk)	21.2.25 (Material and CDK 21.2.14)	Framework and UI components
`@ngx-translate/core`, `http-loader`	^17.0.0	Translations
`chart.js`	^4.5.1	Dashboard and report charts
`jspdf`, `jspdf-autotable`	^4.2.1, ^5.0.8	Export reports to PDF
`xlsx`, `file-saver`	0.20.3 (SheetJS CDN), ^2.0.5	Export to Excel and download files
`iconify-icon`	^3.0.2	Icons
`vitest`, `jsdom` (development)	^4.1.7, ^28.0.0	Unit tests with `@angular/build:unit-test`

2.3 Build and Run

Requirements: Node.js 24 or a version compatible with Angular 21, and npm 11 (`packageManager: npm@11.6.2`). Screens load data only when the backend is running (section 4).

```
cd Web
npm install          # install dependencies
npm start            # ng serve at http://localhost:4200, proxying /api ->
http://localhost:8080
npm run build        # production build in dist/web/browser
npm test             # unit tests (Vitest)
```

The proxy lives in `Web/proxy.conf.json` and is enabled in `angular.json` (`proxyConfig`). The production build has size budgets: it fails if the initial bundle exceeds 1 MB or a component style exceeds 16 kB (`angular.json`, `budgets`). In production, `dist/web/browser` can be served by any static server that forwards `/api` to the gateway.

According to SRS v7 (RNF-008), the web client must work on current versions of Chrome, Firefox, and Edge and be usable from 360 px wide.

3. Mobile Application

The mobile app is built with Expo 57 on React Native 0.86 (Expo, n.d.; React Native, n.d.) and targets Android 8.0 or later (RNF-008 in SRS v6). The web build (`react-native-web`) is used only for development and testing. It talks only to the gateway and discovers the IP of the computer running Expo by itself, so no setup is needed on a local network.

3.1 Structure

Table 6

Folders in `LavadoVehicular-Movil/src`

Folder	Contents
`app/config`	i18next with `es`, `en`, `fr`, `pt` (`app/config/locales`)
`app/providers`	`AppProviders` and account preference sync
`app/theme`	Theme context, colors, and typography
`core/api`	`config.ts` (gateway URL), `httpClient.ts` (fetch with JWT and a 15 s timeout), `apiError.ts`
`core/navigation`	`RootNavigator` (screens per role) and route types
`core/services`	`auth`, `booking`, `vehicles`, `operations`, `payments`, `notifications`, `users`
`core/storage`	Session in `expo-secure-store` (AsyncStorage on web)
`modules/auth`, `client`, `operator`, `admin`	Screens, components, models, hooks, and view models for each role
`shared`	Per-role layouts, shared views (profile, notifications, settings), validators, components, and utilities

3.2 Main Dependencies

Table 7

Mobile libraries (`LavadoVehicular-Movil/package.json`)

Library	Version	Purpose
`expo`	~57.0.19	Build and runtime platform
`react`, `react-native`	19.2.3, 0.86.3	User interface
`@react-navigation/native`, `native-stack`	^7.3.13, ^7.18.5	Navigation
`expo-secure-store`, `@react-native-async-storage/async-storage`	~57.0.4, 2.2.0	Session storage
`i18next`, `react-i18next`	^26.4.1, ^17.0.13	Translations
`expo-print`, `expo-sharing`, `expo-file-system`, `expo-document-picker`	~57.0.x	Export reports and attach receipts
`react-native-svg`, `react-native-reanimated`	15.15.4, 4.5.1	Charts and animations

3.3 Configuration

The backend URL is chosen in this order (`src/core/api/config.ts`):

1. `EXPO_PUBLIC_API_URL` in `.env.local`, if defined.
2. The IP of the computer running Expo (the same one the phone used to open the app).
3. Per platform: `http://10.0.2.2:8080/api/v1` on the Android emulator and `http://localhost:8080/api/v1` on the web build.

The `.env.example` template explains the optional variables; `.env.local` is not committed. The phone and the computer must be on the same Wi-Fi network, and the firewall must allow port 8080.

3.4 Build and Run

```
cd LavadoVehicular-Movil
npm install
npm start                # expo start: scan the QR code with Expo Go
npm run android          # open on an Android emulator or device
npm run web              # web version at http://localhost:8081
npx expo start --clear   # restart and clear the cache after changing .env.local
```

The repository has no `eas.json` and no APK build scripts; the app runs through Expo Go.

4. Backend Required by the Clients

The web and mobile clients only need the gateway at `localhost:8080`; behind it, the six services, SQL Server, and (if events are enabled) RabbitMQ must be running. Everything starts from `Backend/lavarapido-infra` with Docker.

```
cd Backend/lavarapido-infra
cp .env.example .env                          # fill in secrets; .env is never committed
docker compose up -d                          # only SQL Server, Mailpit, and RabbitMQ
(services run from the IDE)
docker compose --profile app up -d --build    # the whole backend in containers
```

Table 8

Variables in `.env.example` (no values)

Group	Variables
Database	`DB_PASSWORD` (required), `DB_NAME`, `DB_USERNAME`, `DB_URL`
Security	`JWT_SECRET` (required), `INTERNAL_API_KEY` (calls between services and automatic assignment), `CORS_ALLOWED_ORIGINS`
Email	`MAIL_ENABLED`, `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_FROM`
Messaging	`MESSAGING_ENABLED`, `RABBITMQ_HOST`, `RABBITMQ_PORT`, `RABBITMQ_USERNAME`, `RABBITMQ_PASSWORD`; `AUTO_ASSIGN_ENABLED=false` (optional) turns off only the automatic assignment
Push	`PUSH_ENABLED` (and `EXPO_ACCESS_TOKEN` in the compose file)

To run a Java service from the console, use its Maven Wrapper (`./mvnw spring-boot:run` inside the service folder) with Java 21 (Spring, n.d.). payment-service runs with `dotnet run --project src/PaymentService.Api` and requires the .NET 8 SDK (Microsoft, n.d.). When starting with `--profile app`, the folders must be named after their repositories (`lavarapido-security-service`, `lavarapido-operation-service`), because that is how the compose file references them.

Table 9

Ports to check that everything is up

Service	Health check URL
API Gateway	`http://localhost:8080/actuator/health`
Java services	`http://localhost:300X/actuator/health` (X = 1, 2, 3, 4, 6) and Swagger at `/swagger-ui.html`
payment-service	`http://localhost:3005/actuator/health`; Swagger at `/swagger` only in the Development environment (`Program.cs:50-55`)
RabbitMQ	Dashboard at `http://localhost:15672`
Mailpit	Inbox at `http://localhost:8025`

With everything up, `lavarapido-infra/e2e` has three Node 22 scripts: the cross-service booking flow (`node --test "e2e/*.test.js"`), the response time measurement (`node e2e/performance.js`) and the check of the web at 360 px wide (`node e2e/width-360.js`, after `npm install --prefix e2e`). They need `MESSAGING_ENABLED=true` and `INTERNAL_API_KEY` in the `.env`.

References

Angular. (n.d.). *Angular documentation*. Google. Retrieved October 9, 2026, from https://angular.dev/

Expo. (n.d.). *Expo documentation*. Retrieved October 9, 2026, from https://docs.expo.dev/

Microsoft. (n.d.). *.NET documentation*. Retrieved October 9, 2026, from https://learn.microsoft.com/dotnet/

React Native. (n.d.). *React Native documentation*. Meta Platforms. Retrieved October 9, 2026, from https://reactnative.dev/docs/getting-started

Spring. (n.d.). *Spring Boot reference documentation*. Broadcom. Retrieved October 9, 2026, from https://docs.spring.io/spring-boot/